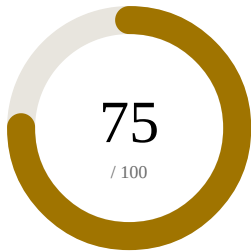


Sample Co (MySQL)

Engine: **MySQL** 8.0.46 · Collected: 05 Jul 2026 13:01 UTC · Database size: 855.1 MB



Health score: 75

queries -17, indexes -7, config -1

Executive summary

- Overall health score: 75/100 — degraded; the issues below cost real performance.
- 8 findings in total: 1 critical, 2 medium, 4 low, 1 info.
- Biggest issue: one query consumes a large share of total database time (SELECT `id` , STATUS , `total\_cents` FROM `orders` WHERE `customer\_id` = ?).
- 3 index findings — usually the cheapest wins in this report.
- 2 configuration settings worth reviewing; each cites its current value and a recommended range.

Top 5 issues

CRITICAL R-Q1

1. One query consumes a large share of total database time

```
SELECT `id` , STATUS , `total_cents` FROM `orders` WHERE `customer_id` = ?
```

calls	541
total_ms	237165.8
pct_of_db_time	69.1
mean_ms	438.38
rows_per_call	1.5
query_digest	8cc5472ae3d83cfec6094b85fd7bcf412e9362fc184db7e8e8806fdd93614d8b

This single statement dominates database time. Investigate its execution plan first; typical fixes are an index on its filter columns, caching its result if it repeats, or restructuring the query. Test any change in staging first.

► [Copy as ticket \(GitHub / Jira markdown\)](#)

MEDIUM R-I1

2. Table is scanned in full by frequent queries (missing index candidate)

orders

table_size	114.7 MB
candidate_columns	customer_id
calls	541
rows_examined_per_call	2000000
rows_returned_per_call	1.5

Queries filter orders on (customer_id) without a usable index, so the table is read in full each time. Consider an index on (customer_id). Test in staging first: measure the query with EXPLAIN before and after.

⚠ Confidence: medium — test this change in staging before applying it to production.

► [Copy as ticket \(GitHub / Jira markdown\)](#)

MEDIUM R-I3

3. Redundant index duplicates another index's leading columns

shop.users.users_email_idx

redundant_index	(email)
covered_by	sys.schema_redundant_indexes

Index users_email_idx is covered by another index with the same leading columns; it adds write cost and space for no read benefit. Verify with usage stats over a full business cycle, then drop the redundant one. Test in staging first: measure the query with EXPLAIN before and after.

► [Copy as ticket \(GitHub / Jira markdown\)](#)

LOW R-Q2

4. Query is slow on every execution

SELECT `SLEEP` (?)

mean_ms	8000.4
calls	6
total_ms	48002.4
rows_per_call	1.0
query_digest	3b74a4bb71311c8dfffd4b64a1dabe02d0ac8fcd7ae03087dd69378a554bb28bc

Every execution takes over a second. Check the execution plan for full scans or sorts; an index on the filter/sort columns is the usual fix. If the result changes rarely, cache it. Test in staging first.

► [Copy as ticket \(GitHub / Jira markdown\)](#)

LOW R-Q2

5. Query is slow on every execution

UPDATE `users` SET NAME = ? WHERE `id` = ?

mean_ms	3500.8
calls	12
total_ms	42010.2
rows_per_call	0.0
query_digest	c5794024ae9adc9e88bc2d57764781b2b4042b9c8f947a6acda59d2adef219e2

Every execution takes over a second. Check the execution plan for full scans or sorts; an index on the filter/sort columns is the usual fix. If the result changes rarely, cache it. Test in staging first.

► [Copy as ticket \(GitHub / Jira markdown\)](#)

Slow queries

Statement (normalized)	Calls	Total ms	Mean ms	% of DB time
SELECT `id` , STATUS , `total_cents` FROM `orders` WHERE `customer_id` = ?	541	237,166	438.38	69.1
SELECT `SLEEP` (?)	6	48,002	8000.40	14.0
UPDATE `users` SET NAME = ? WHERE `id` = ?	12	42,010	3500.85	12.2
SELECT `id` , `user_id` , `kind` FROM EVENTS ORDER BY `id` LIMIT ? OFFSET ?	541	12,083	22.34	3.5
SELECT `product_id` , `quantity` , `price_cents` FROM `order_items` WHERE `order_id` = ?	10,820	1,977	0.18	0.6
SELECT `id` FROM `orders` WHERE `id` > ? ORDER BY `id` LIMIT ?	541	950	1.76	0.3
SELECT `id` , `email` FROM `users` WHERE `id` = ?	5,410	888	0.16	0.3
ROLLBACK	12	34	2.82	0.0
SHOW GLOBAL VARIABLES	2	5	2.46	0.0
SELECT `object_name` , `index_name` FROM `sys` . `schema_unused_indexes` WHERE `object_schema` = ?	1	3	3.45	0.0

Top statements by cumulative execution time since statistics were last reset. All literals are stripped before data leaves your environment.

Index advisor

Severity	Finding	Object	Evidence
MEDIUM	Table is scanned in full by frequent queries (missing index candidate)	orders	table_size=114.7 MB; candidate_columns=customer_id; calls=541; rows_examined_per_call=2000000; rows_returned_per_call=1.5
MEDIUM	Redundant index duplicates another index's leading columns	shop.users.users_email_idx	redundant_index=(email); covered_by=sys.schema_redundant_indexes
LOW	Index is never used but costs	shop.users.users_email_dup_idx	source=sys.schema_unused_indexes (since server start); definition=(email)

space and write
time

Growth & operational risk

Connections at collection time: 1 (peak since restart: 4) of 151 allowed.

No lock, connection, or growth risks detected on this run.

InnoDB health (MySQL)

Table	Size	Reclaimable (data_free)	Rows (est.)
shop.order_items	370.5 MB	7.0 MB	3,985,964
shop.users	205.1 MB	5.0 MB	996,499
shop.events	164.7 MB	7.0 MB	1,991,368
shop.orders	114.7 MB	5.0 MB	1,993,434

Configuration findings

LOW innodb_log_file_size

innodb_log_file_size 48.0 MB

Small redo logs force frequent checkpoints; on write-heavy workloads that shows up as periodic stalls. Typical modern values are 512MB-2GB. Suggested change: Raise innodb_log_file_size (requires restart; on 8.0.30+ use innodb_redo_log_capacity).

INFO slow_query_log

slow_query_log OFF

The slow query log is off. performance_schema covers aggregate statistics, but the log captures concrete worst-case executions — useful corroborating evidence when chasing intermittent slowness. Suggested change: Enable slow_query_log with long_query_time ~1s; rotate the file.

Developer action plan

1. [Top issue 1] One query consumes a large share of total database time — `SELECT `id` , STATUS , `total_cents` FROM `orders` WHERE `customer_id` = ?`. This single statement dominates database time.

2. [Top issue 2] Table is scanned in full by frequent queries (missing index candidate) — orders. Queries filter orders on (customer_id) without a usable index, so the table is read in full each time.
3. [Top issue 3] Redundant index duplicates another index's leading columns — shop.users.users_email_idx. Index users_email_idx is covered by another index with the same leading columns; it adds write cost and space for no read benefit.
4. [Top issue 4] Query is slow on every execution — SELECT `SLEEP` (?). Every execution takes over a second.
5. [Top issue 5] Query is slow on every execution — UPDATE `users` SET NAME = ? WHERE `id` = ?. Every execution takes over a second.
6. Re-run the collector after each change to verify the effect before the next one.

Every recommendation above should be validated in a staging environment first. Capabilities on this run: table_stats, index_stats, sessions, config, lock_waits, query_stats, sys_schema. Notes: 15 statement instruments are disabled; some statements may be missing from query statistics.